Universitetet i Sørøst-Norge

ADAptive

Ada-based control of a Mecanum vehicle

VE3140 – Intelligent Real-Time Systems

Project report

Jacob Hellumsand Utengen
Nicolay Olafsen

Semester/year: to be confirmed

Phase 2 working draft

https://github.com/kidi970/ADAptive-VE3140

Abstract

Provisional abstract; the evaluation and conclusion remain incomplete.

ADAptive is a student project in VE3140 that aims to develop an Ada-based real-time embedded controller for a four-wheel Mecanum vehicle using a BBC micro:bit v2. The intended system combines front and rear ultrasonic sensing with control logic and a DFR0548 motor driver. The report will document the requirements, hardware and software architecture, implementation, and timing analysis. Functional tests and timing measurements will provide the basis for evaluating the prototype. This draft establishes the project context, sensor and vehicle background, proposed control rules, and an evaluation method. Source inspection identifies driver behaviour that must be checked before timing bounds can be established. Autonomous operation and timing requirements remain to be validated.

Contents

Abstract **i**

1 Introduction **1**

2 Requirements and project objectives **2**

3 Background **3**

4 System design **5**

5 Real-time analysis **7**

6 Implementation **9**

7 Testing and results **9**

8 Discussion and evaluation **10**

9 Project management and collaboration **10**

10 Conclusion **10**

References **11**

A Essential Ada code **12**

1 Introduction

1.1 Motivation and problem context

A small mobile robot provides a practical setting for studying the interaction between software, hardware, and timing. A movement command must correspond to the physical direction of each wheel, while a distance measurement must be available early enough to influence that command. A correct decision based on an outdated measurement may still produce an unsuitable response. ADAptive uses this relationship to investigate real-time control on a resource-limited embedded platform.

The VE3140 project brief requires a functioning real-time system programmed in Ada, using a BBC micro:bit v2, associated sensors, and a Mecanum car [1]. The group consists of Jacob Hellumsand Utengen and Nicolay Olafsen. The selected platform connects the micro:bit to a DFR0548 motor driver and four wheel motors. Front and rear ultrasonic sensors are being developed as inputs for obstacle detection. These components make both the physical interfaces and the timing of the sensing-to-actuation path central to the project.

1.2 Objective and research question

The objective is to develop and evaluate a prototype that uses Ada to acquire distance information and control the vehicle through explicit movement states. The intended behaviour includes forward motion, stopping, and reversing in response to the surrounding obstacles. These behaviours are development objectives; their integration and reliability must be supported by code and tests before they are treated as demonstrated results.

The project is guided by the following question: *How can an Ada-based controller on a BBC micro:bit v2 coordinate ultrasonic sensing and Mecanum motor control, and what evidence is needed to show that its functional and timing requirements are met?* The evaluation will therefore consider both observable vehicle behaviour and the time required to acquire inputs, make decisions, and issue motor commands.

1.3 Scope and real-time character

The scope covers the controller, sensor interfaces, motor control, development workflow, and validation of the prototype. The initial autonomous behaviour is limited to obstacle-responsive movement; general route planning, localisation, and a complete omnidirectional navigation system are outside this initial scope. The use of Mecanum wheels does not itself establish that lateral motion or a full kinematic controller has been implemented.

ADAptive is an embedded system because its software runs on a microcontroller and interacts directly with sensors and actuators. Its intended control function is real-time because the usefulness of an obstacle response depends on when it occurs, as well as on the selected movement. Numerical deadlines must be derived from the operating conditions and then checked against implementation timing. No hard real-time guarantee is asserted in this draft.

1.4 Report organisation

Section 2 defines the initial objectives and requirements. Sections 3 and 4 introduce the technical background and system overview. The later report will develop the timing analysis in Section 5, followed by implementation, testing and results, discussion, project management, and conclusions in Sections 6–10. Selected Ada code will be included in Appendix A.

2 Requirements and project objectives

Table 1 separates the assignment constraints from the group's intended behaviour and proposed timing and fail-safe requirements. The latter are an initial specification for review, rather than evidence of implementation. Acceptance tests and all numerical bounds remain to be agreed. In particular, a motor stop command must not be equated with an instantaneous physical stop.

Table 1: Initial requirements and planned verification.

ID	Requirement	Basis and verification
Functional requirements		
F1	Use Ada on a BBC micro:bit v2 to control a Mecanum vehicle with associated sensors.	Assignment. Demonstrate the integrated prototype.
F2	Command the four wheel motors through the DFR0548, using the group's verified motor mapping.	Group objective. Record individual motor and vehicle movement tests.
F3	Acquire front and rear ultrasonic readings and make their validity available to the controller.	Group objective. Test both channels against reference distances and missing echoes.
F4	Use explicit control states for forward motion, stopping, and reversing; define the conditions for each transition.	Group objective. Review a state diagram and test its transitions.
Real-time requirements (proposed)		
RT1	Bound the age of sensor data used in a movement decision. Maximum age: *TODO*.	Agree acquisition interval and timestamp semantics; measure data age.
RT2	Bound the delay from a relevant obstacle observation to the resulting motor command. Deadline: *TODO*.	Derive a bound from operating conditions; measure the complete response path.
RT3	Define and justify periods, priorities, deadlines, and execution-time bounds for the final schedulable entities.	Establish the task model and apply suitable timing analysis. Values: *TODO*.
Safety and fail-safe requirements (proposed)		
S1	Keep motor outputs in a defined stop state during initialisation, until the conditions for movement are satisfied.	Review startup logic and test startup and reset behaviour.
S2	Inhibit movement towards an obstacle when the relevant reading is missing, invalid, or too old; request a defined stop state.	Agree validity rules and inject missing, invalid, and stale readings.
S3	Bound sensor waiting so that a missing echo cannot indefinitely prevent a control response. Timeout: *TODO*.	Inspect the selected driver and test echo timeout paths.
Development and documentation requirements		
D1	Deliver a functioning prototype and a report of approximately 15–20 pages, excluding appendices.	Assignment. Review the final demonstration and report.
D2	Maintain source and build information in the GitHub repository; identify the revision used for evaluation.	Documentation objective. Reproduce the build from the recorded revision.
D3	Support functional and timing claims with code, test records, and measurements, and retain essential Ada code in an appendix.	Evidence rule. Link claims to test records and the evaluated source revision.

3 Background

3.1 BBC micro:bit v2

The BBC micro:bit v2 uses a Nordic nRF52833 application processor with an Arm Cortex-M4 core, operating at 64 MHz, with 512 kB of flash memory and 128 kB of RAM. Its edge connector exposes GPIO for external hardware, while the board includes user buttons and a 5×5 LED display [2]. In ADAptive, the application processor is intended to run the sensor acquisition and control software. The external expansion board provides the motor interfaces.

The board also has a separate interface processor for USB communication, flashing, and debug access. The debug connection to the application processor uses Serial Wire Debug [2]. This distinction matters when documenting the development setup: the computer and debug tools support development, while the vehicle controller executes on the micro:bit itself. The inspected repository provides a PyOCD-based debug configuration, but this configuration alone does not establish a successful hardware debug session [3].

3.2 Embedded real-time control

An embedded controller connects program execution to physical events. For ADAptive, this means reading the distance sensors, selecting a movement state, and sending commands to the motor driver. The controller must treat the time of a measurement as part of its meaning: a reading that was valid when acquired may no longer describe the space in front of a moving vehicle.

A deadline specifies when an operation must complete relative to a defined event. A period specifies the interval between releases of a periodic task. Worst-case execution time (WCET) is a bound on the processor time required by that task, whereas response time also includes interference from other execution. Fixed-priority response time analysis uses these quantities to evaluate whether task deadlines can be met [4].

For this prototype, software timing and physical stopping must be distinguished. A deadline for issuing a stop command does not establish how far the vehicle travels before it stops. The final requirements must relate sensing delay, control delay, and physical behaviour to the tested speed and environment. Those bounds are not yet available.

3.3 Ada, runtime profiles, and drivers

Ada provides typed interfaces, packages, and language-level concurrency mechanisms that can support the separation of sensing and motor control. Ravenscar and Jorvik are tasking profiles defined by the Ada standard. Both specify fixed-priority dispatching and ceiling locking, together with restrictions on tasking constructs. Jorvik relaxes some restrictions imposed by Ravenscar [5]. Their relevance to ADAptive is the possibility of describing a bounded set of activities and shared resources that can be analysed. Selecting a profile does not, on its own, prove that deadlines are met.

The inspected application project selects the runtime `embedded-nrf52833`, enables Ada 2022 syntax, and imports `microbit_v2_full.gpr`. The driver's generated project metadata contains the label `ravenscar-full`; the exact effective runtime profile must still be confirmed from the toolchain and build configuration [3, 6]. This draft therefore does not identify the application as a verified Ravenscar or Jorvik task set.

The pinned Ada Drivers Library provides `MicroBit.MotorDriver`, `MicroBit.Ultrasonic`, and a DFR0548 component driver. Their presence does not establish application integration [6].

3.4 Ultrasonic distance sensing

An ultrasonic range sensor estimates distance from the round-trip travel time of an acoustic pulse. For an echo pulse width Δt representing this travel time, the ideal distance is

$$d = \frac{c\,\Delta t}{2}, \tag{1}$$

where c is the speed of sound in the surrounding air. The factor of two accounts for travel to the reflecting surface and back. Sound speed and the quality of the returned echo affect the estimate; this equation does not establish the accuracy of the installed sensors.

The group reports a rear HC-SR04. Its datasheet specifies a trigger pulse of at least $10\,\mu$s, an eight-cycle 40 kHz burst, and an echo pulse whose width represents range. It recommends a measurement cycle longer than 60 ms [7]. These are datasheet values, not ADAptive measurements or selected task periods. The front sensor model and the final sensor revisions must still be identified.

Echo quality depends on the reflecting surface, its orientation, and the measurement conditions. The HC-SR04 datasheet specifies a smooth target and a substantial reflecting area [7]; a reliable response to that target does not establish detection of every object encountered by the car. The proposed acquisition method triggers only one sensor at a time and leaves an acoustic guard interval before the next trigger. This reduces the risk of attributing one sensor's transmission to the other, but the interval and effectiveness must be verified on the prototype.

An absent echo, an out-of-range return, and a usable distance are distinct outcomes. Treating all three as a distance of zero loses information needed by the controller. Each reading should therefore carry a validity status and a timestamp, as proposed in Section 4.3.

The HC-SR04 datasheet specifies a 5 V supply, while micro:bit v2 GPIO input voltages are limited relative to the processor supply [7, 8]. Sensor supply voltage and echo output voltage must be checked separately. The report cannot assume that a particular level converter or voltage divider is fitted, or that direct echo wiring is suitable.

3.5 Mecanum vehicle and motor commands

Mecanum wheels use angled passive rollers [9]. With a suitable wheel arrangement, the four driven wheel speeds can combine to produce forward, lateral, and rotational motion. Kinematic descriptions relate chassis velocity to individual wheel velocities and require the wheel locations and coordinate convention [10].

For ADAptive, the physical mapping in Table 2 takes precedence over software names such as `rf` and `lb`. A positive motor command is defined here as the direction that the group verified as physically forward for that wheel. Forward and reverse baseline tests should therefore command all wheels in the same physical direction, while checking that each channel produces the expected motion. Unequal wheel response can still cause drift.

The motor driver accepts PWM commands; these are not measured wheel speeds. A full velocity-based kinematic controller would additionally require the wheel arrangement, geometry, and a validated relationship between the command and wheel motion. That controller is outside the current Forward/Stop/Reverse design. Roller orientation, wheel dimensions, and any feedback sensors remain undocumented, so no numerical kinematic model is assigned to the prototype.

4 System design

4.1 Initial system overview

Figure 1 shows the intended signal and command path. The control logic and finite state machine (FSM) execute on the micro:bit; they are not separate hardware. The DFR0548 supplies four motor channels [11]. The group identifies the expansion board as the Micro:bit Driver Expansion Board V2.0. The overview does not prescribe a task decomposition or claim that this complete path is implemented in the inspected application.

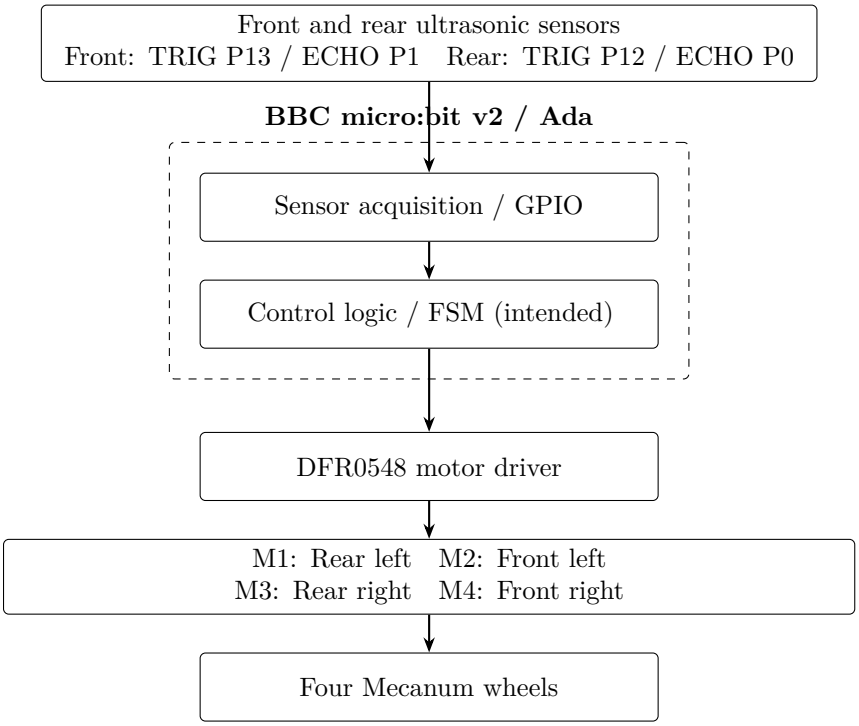


Figure 1: Intended ADAptive architecture. Sensor assignments and wheel mapping are supplied by the group. Arrows represent sensing and commands; power wiring and detailed electrical interfaces are omitted.

Interface	Assignment	Evidence status
Front ultrasonic	TRIG: P13; ECHO: P1	Group-supplied setup; under development.
Rear ultrasonic	TRIG: P12; ECHO: P0	Group-supplied setup; under development.
Motor channels	M1: rear left; M2: front left; M3: rear right; M4: front right	Individual forward directions reported as physically verified by the group.

Table 2: Initial hardware assignments. Motor verification is reported by the group; test records have not yet been supplied.

4.2 Implementation evidence boundary

At the inspected repository revision `3ba92c0`, `Code/src/main.adb` polls button A, button B, and the logo input, updates the display, and writes console messages. It does not contain the vehicle FSM, motor commands, or ultrasonic acquisition [3]. The group's account describes later motor, distance-measurement, and Forward/Stop/Reverse experiments [1]. These are retained as reported development context, pending the corresponding source revision and test records. The complete motor and sensor integration remains unverified within this report draft.

4.3 Proposed measurement contract

The following interface is a design proposal for review. Each sensor reading should contain a distance, a validity status, and an acquisition timestamp. The distance is used only when its status is valid. Recording acquisition start as the timestamp gives a conservative age that includes the time spent waiting for the echo; the final implementation must use one documented convention. At decision time t, a reading is fresh only if

$$0 \leq t - t_{\text{acq}} \leq A_{\max}, \tag{2}$$

where $A_{\max}$ is the agreed maximum age, currently *TODO*. Front and rear readings are evaluated separately because sequential acquisition gives them different ages. Re-reading a stored distance must not refresh its timestamp.

The acquisition layer should distinguish timeout and invalid-range outcomes from valid readings. The controller then checks the sensor relevant to the requested direction: front for forward motion and rear for reverse motion. This is the proposed behaviour behind requirements F3, RT1, and S2; it is not present in the inspected main program.

4.4 Proposed movement states

Table 3 defines a candidate state policy consistent with the reported Forward/Stop/Reverse experiments. It does not reconstruct the group's unprovided FSM. An explicit enable condition, obstacle threshold, reversal policy, and recovery procedure remain to be agreed.

State	Condition for movement	Proposed response to loss of permission
Stop	Motors receive the defined stop command. Movement requires enable and fresh, valid clearance in the requested direction.	Remain stopped while required input is invalid or movement is disabled.
Forward	Front reading is fresh and valid, and above the agreed obstacle threshold.	Enter Stop if the front path is blocked, its reading is invalid or stale, or movement is disabled.
Reverse	Rear reading is fresh and valid, and above the agreed obstacle threshold.	Enter Stop if the rear path is blocked, its reading is invalid or stale, or movement is disabled.

Table 3: Candidate movement-state policy; not an implemented or tested FSM.

Forward-to-reverse and reverse-to-forward transitions should pass through Stop. A front obstacle does not, by itself, permit reversing: rear clearance must also be established. The necessary dwell time, if any, and conditions for resuming motion remain open. A software stop request is a commanded output, not a claim that the driver brakes the vehicle or that all faults can stop it.

4.5 Functional responsibilities

Acquisition should produce readings with explicit status and age information; control should evaluate guards and select a state; actuation should translate that state into commands for the verified motor mapping. This separation defines interfaces rather than a task allocation. The functions may execute in one bounded cycle or in separately scheduled tasks, depending on the final design. The chosen arrangement must preserve a timely stop response when a sensor or motor transaction fails.

5 Real-time analysis

5.1 Task and measurement template

Table 4 is reserved for the final task set. The inspected application uses a polling loop and does not define a schedulable vehicle task set. No sensor or control function is assigned a separate task in this draft. All values must refer to the evaluated source revision and build configuration.

Task	Priority	Period T_i	Deadline D_i	WCET C_i	Response R_i
TODO: identify	*TODO*	*TODO*	*TODO*	*TODO*	*TODO*
TODO: extend	*TODO*	*TODO*	*TODO*	*TODO*	*TODO*

Table 4: Timing template. Rows are placeholders, not implemented tasks. Time units and priority ordering are to be specified.

5.2 Response time analysis method

The standard basic fixed-priority response time equation is [4, 12]

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j. \tag{3}$$

Here $hp(i)$ is the set of tasks with priorities higher than task i; C_i is its execution-time bound and T_j is the period or minimum inter-arrival time of task j. The least fixed point is normally sought by iteration starting at C_i, and compared with the relative deadline D_i.

This basic form assumes a single processor, fixed-priority preemptive scheduling, independent tasks, constrained deadlines ($D_i \leq T_i$), no release jitter, and no unmodelled blocking or self-suspension. Relevant overheads and interrupt execution must be accounted for. Its applicability to ADAptive must be checked against the final implementation before any numerical analysis is performed. End-to-end obstacle response must also include acquisition and actuation delays; meeting an individual task deadline is not sufficient evidence for that whole path.

5.3 Execution-time measurement plan

Before filling Table 4, the evaluated firmware revision, runtime, compiler options, and instrument must be recorded. Acquisition tests should exercise both normal echoes and fault paths, while control tests should exercise each state transition. Motor transactions and any shared-resource access must also be included. Interrupt and runtime activities cannot be excluded merely because they are absent from the application's main file.

GPIO markers observed with a logic analyser, or a suitable on-board timer, can define the start and completion of each activity. The selected method must have adequate resolution and its overhead must be characterised. Console output should be outside the measured interval unless it is part of the final timing-critical operation. Measurements taken while execution is halted in a debugger do not represent continuous operation.

Elapsed time measured around a call may contain preemption or waiting, and is not automatically that call's processor execution time. In particular, the pinned ultrasonic driver uses polling and busy-wait delays. Its source timeout constants must not be entered as C_i: the source inspection in Section 6.2 does not establish termination for every echo condition. Observed maxima, measurement conditions, and the argument for a conservative execution bound should be reported separately.

5.4 Deadline derivation and analysis procedure

The obstacle response requirement must begin at a defined event. Timing from a completed sensor reading to a motor command omits the delay until the next reading. For the physical obstacle response, account for sampling phase, acquisition, waiting for control execution, decision processing, and delivery of the motor command. The physical stopping distance must be evaluated separately at the chosen speed. This evidence is needed to justify RT2's deadline and the obstacle threshold rather than selecting convenient values.

Once the actual scheduling model is established, Equation 3 can be evaluated by

$$r_i^{(0)} = C_i, \qquad r_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{r_i^{(k)}}{T_j} \right\rceil C_j. \tag{4}$$

Iteration stops at a fixed point or when the deadline is exceeded [12]. A fixed point satisfying $R_i \leq D_i$ supports schedulability only under the stated assumptions and justified input bounds. Blocking, release jitter, or suspension require a suitable extended model. If the design remains a single polling loop, analyse its cycle and longest input-to-output path instead of treating its functions as preemptive tasks.

6 Implementation

6.1 Verified source and build configuration

The inspected application remains a button/display prototype. Its repository contains `Code/src/main.adb`, the project file `Code/itrs.gpr`, the toolchain description `alire.toml`, and build/flash tooling in `tools/mb.py` [3]. The project selects the `arm-eabi` target and `embedded-nrf52833` runtime. Alire pins `gnat_arm_elf` to 15.1.2 and `gnatprove` to 15.1.0; their presence does not establish that a firmware build or proof has passed. The newer vehicle implementation is not available in the inspected source.

The pinned library's `MicroBit.MotorDriver` uses the external I^2C controller and a DFR0548 component driver to write wheel PWM commands [6]. The logical wheel arguments must be traced to physical ports and checked against Table 2 before adopting movement patterns. A library direction name is not a physical calibration result.

6.2 Static driver inspection

The library's `MicroBit.Ultrasonic` accepts trigger and echo GPIO points as generic parameters. Its acquisition path requests a nominal 10 μs trigger delay, then passes 3 ms and 23 ms constants to the echo-start and echo-end polling routines [6]. These are source-level settings, not measured timing bounds.

Both routines have an important termination limitation. On deadline expiry, the echo-start routine sets a timeout flag and calls `Echo.Clear`; the echo-end routine resets the distance and calls `Echo.Set`. Neither branch exits or returns. The loop exit conditions still depend on the echo input. In `nRF.GPIO`, the Boolean `Set` function reads the input register, whereas the `Set`/`Clear` procedures write the output register [13]. Thus changing the output latch of the configured input does not establish an exit for an echo held low or high. The comments about maximum waiting time are insufficient to justify a finite acquisition bound. This is a finding from code inspection; its occurrence on ADAptive has not been tested.

The distance conversion also counts polling iterations containing `Delay_Us(58)` rather than timestamping both echo edges. Loop overhead and preemption therefore affect its relationship to pulse duration [6]. Finally, `Set_PWM_Wheels` obtains an I^2C status without checking it in that routine. It cannot be credited with verified fault recovery from a failed motor transaction. These points motivate the targeted tests in Section 7; no driver was modified in this phase.

7 Testing and results

7.1 Planned test method

Table 5 defines tests to collect evidence for the requirements. It is a test plan, not a record of completed tests. Each record should identify the firmware and driver revisions, hardware and supply configuration, initial state, stimulus, reference measurement, observed behaviour, and acceptance criterion. Repetitions and numerical tolerances remain *TODO*.

Motor tests should first confirm the physical channel mapping, then check whole-vehicle movement. Sensor tests should compare both channels with a known reference distance and retain invalid outcomes as well as usable readings. The group-reported sensor instability and rear HC-SR04 heating incident should be documented from contemporaneous observations; they do not establish a particular hardware or software cause.

Table 5: Planned tests and required evidence. All tests remain pending.

Test	Stimulus and requirement	Evidence to record
T1	Command each motor, then forward/reverse/stop on the vehicle. F1–F2.	Port, physical direction, command, motion, and stopping behaviour.
T2	Test front and rear sensing at reference distances and with different target orientations. F3.	Raw readings, reference distance, invalid returns, repetitions, and variation.
T3	Exercise absent echo and echo held high/low using a documented, electrically compatible setup. F3, S3.	Whether the acquisition returns, returned status, elapsed time, and independent trace.
T4	Exercise each proposed state guard, including expired timestamps and loss of clearance while moving. F4, RT1, S2.	Input value/status/age, previous and next state, and issued motor command.
T5	Startup, reset, and loss of movement permission. S1–S2.	Initial motor outputs and transitions before permission is established.
T6	Introduce obstacles in each movement direction; test sequential operation of both sensors. RT2, F3–F4.	Sampling phase, acquisition/decision/command times, speed, stop distance, and cross-sensor behaviour.
T7	Exercise the relevant execution paths and motor transaction failures. RT3, S2–S3.	Instrumentation, execution observations, interrupt/waiting contributions, and fault response.

7.2 Results awaiting evidence

No test records or timing measurements have been supplied for this draft. Test outcomes, requirement pass/fail decisions, and numerical RTA remain open. The group's reported individual motor checks can be incorporated when their setup and observations are recorded; they are not replaced by the pending tests in Table 5.

8 Discussion and evaluation

Reserved for a later phase: interpretation of results, limitations, and reflection.

9 Project management and collaboration

Reserved for a later phase: the group's planning, division of work, and collaboration.

10 Conclusion

Reserved for a later phase: an evidence-based answer to the research question.

References

[1] ADAptive project group, "ADAptive: project brief and current hardware/development description," Unpublished material supplied by the project group, includes the VE3140 assignment wording; hardware observations are group-reported.

[2] Micro:bit Educational Foundation, "BBC micro:bit hardware documentation," https://tech.microbit.org/hardware/, accessed 8 October 2026.

[3] kidi970, "ADAptive-VE3140: source repository," https://github.com/kidi970/ADAptive-VE3140/tree/3ba92c01d4720299d71bb35cb840fc93f028ef47, inspected main-branch commit 3ba92c0; accessed 8 October 2026.

[4] M. Joseph and P. Pandya, "Finding response times in a real-time system," *The Computer Journal*, vol. 29, no. 5, pp. 390–395, 1986. [Online]. Available: https://doi.org/10.1093/comjnl/29.5.390

[5] Ada Rapporteur Group, "Ada 2022 Reference Manual, d.13: The Ravenscar and Jorvik profiles," https://ada-rapporteur-group.github.io/ARM/Ada_2022/RM-D-13.html, accessed 8 October 2026.

[6] AdaCore and Ada Drivers Library contributors, "Ada Drivers Library: project-pinned fork," https://github.com/aiunderstand/Ada_Drivers_Library/tree/8e90f85575666684cc6e0b51830494de259bd3bc, inspected submodule commit 8e90f85; accessed 8 October 2026.

[7] ELECFREAKS, "Ultrasonic ranging module HC-SR04," https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf, module datasheet hosted by SparkFun; accessed 8 October 2026.

[8] Micro:bit Educational Foundation, "Edge connector and micro:bit pinout," https://tech.microbit.org/hardware/edgeconnector/, v2 pin assignments and nRF52 GPIO limits; accessed 8 October 2026.

[9] REV Robotics, "Mecanum wheel setup and behavior," https://docs.revrobotics.com/duo-build/mecanum-drivetrain-v2/mecanum-wheel-setup-and-behavior, manufacturer's drivetrain explanation; accessed 8 October 2026.

[10] WPILib contributors, "Mecanum drive kinematics," https://docs.wpilib.org/en/stable/docs/software/kinematics-and-odometry/mecanum-drive-kinematics.html, fIRST Robotics Competition documentation; accessed 8 October 2026.

[11] DFRobot, "micro:Driver: Driver expansion board, DFR0548," https://wiki.dfrobot.com/dfr0548/, accessed 8 October 2026.

[12] S. Altmeyer, R. I. Davis, and C. Maiza, "Improved pre-emption cost aware response time analysis for fixed priority pre-emptive systems," *Real-Time Systems*, vol. 48, no. 5, pp. 499–526, 2012, author manuscript; Section 3 describes the baseline response time equation. [Online]. Available: https://www-users.york.ac.uk/~rd17/papers/ImpCRPD.pdf

[13] AdaCore and Ada Drivers Library contributors, "nRF.GPIO: input reads and output writes," https://github.com/aiunderstand/Ada_Drivers_Library/blob/8e90f85575666684cc6e0b51830494de259bd3bc/arch/ARM/Nordic/drivers/nrf_common/nrf-gpio.adb, inspected pinned dependency source; accessed 8 October 2026.

A Essential Ada code

Reserved for a later phase: selected excerpts from the evaluated application revision.